

HPC -LAB-07

: [by Gagan \(U23AI060\)](#)

PROGRAM -01:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 4

int main()
{
    int A[N][N], B[N][N], C_seq[N][N], C_par[N][N];
    int i, j, k;

    double start, end;

    // Initialize matrices
    for(i = 0; i < N; i++)
    {
        for(j = 0; j < N; j++)
        {
            A[i][j] = i + j;
            B[i][j] = i * j;

            C_seq[i][j] = 0;
            C_par[i][j] = 0;
        }
    }

    // Sequential Matrix Multiplication
    start = omp_get_wtime();

    for(i = 0; i < N; i++)
    {
        for(j = 0; j < N; j++)
        {
            for(k = 0; k < N; k++)
            {
                C_seq[i][j] += A[i][k] * B[k][j];
            }
        }
    }
```

```

}

end = omp_get_wtime();

printf("Sequential Execution Time = %f seconds\n", end - start);

// Parallel Matrix Multiplication
start = omp_get_wtime();

#pragma omp parallel for private(j,k)
for(i = 0; i < N; i++)
{
    for(j = 0; j < N; j++)
    {
        for(k = 0; k < N; k++)
        {
            C_par[i][j] += A[i][k] * B[k][j];
        }
    }
}

end = omp_get_wtime();

printf("Parallel Execution Time = %f seconds\n", end - start);

// Print Result Matrix
printf("\nResult Matrix:\n");

for(i = 0; i < N; i++)
{
    for(j = 0; j < N; j++)
    {
        printf("%d ", C_par[i][j]);
    }
    printf("\n");
}

return 0;
}

```

```

(base) gagan747@gaganLaptop:/mnt/d/Semester_06/HPC/LAB/07_lab_$ gcc -fopenmp q1_matrix_mult.c -o p1
(base) gagan747@gaganLaptop:/mnt/d/Semester_06/HPC/LAB/07_lab_$ ./p1
Sequential Execution Time = 0.000001 seconds
Parallel Execution Time = 0.007765 seconds

```

Analysis of the code:

"In this program, we implemented matrix multiplication using both sequential and parallel approaches. In the sequential method, computation is performed using nested loops where a single processor executes all operations step by step.

In the parallel approach, the outer loop is parallelized using OpenMP (`#pragma omp parallel for`), which distributes rows among multiple threads.

The parallel version reduces execution time because multiple threads compute different rows simultaneously. The correctness of the program is verified by comparing both results."

Program _02 :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4
5  #define N 500
6
7  int main()
8  {
9      static int A[N][N], B[N][N], C[N][N];
10
11     int i, j, k;
12
13     double start, end;
14
15     // Initialize matrices
16     for(i = 0; i < N; i++)
17     {
18         for(j = 0; j < N; j++)
19         {
20             A[i][j] = 1;
21             B[i][j] = 1;
22             C[i][j] = 0;
23         }
24     }
25
26     start = omp_get_wtime();
27
28     #pragma omp parallel for collapse(2) private(k)
29     for(i = 0; i < N; i++)
30     {
31         for(j = 0; j < N; j++)
32         {
33             for(k = 0; k < N; k++)
34             {
35                 C[i][j] += A[i][k] * B[k][j];
36             }
37         }
38     }
39
40     end = omp_get_wtime();
41
42     printf("Execution Time with collapse(2) = %f seconds\n", end - start);
43
44     printf("Sample Output C[0][0] = %d\n", C[0][0]);
45
46     return 0;
47 }
48
49 // Analysis of the code:
50 // "In this program, matrix multiplication is implemented using the OpenMP collapse clause.
51 // The collapse(2) directive combines two nested loops into a single loop for parallel execution.
52 // This improves load balancing among threads compared to parallelizing only one loop.
53
54 // Each thread computes multiple elements of the result matrix, leading to better
55 // utilization of CPU resources and improved performance."
```

```
(base) gagan74@gaganlaptop:/mnt/d/Semester_06/MPC/LAB/07_lab_$ gcc -fopenmp q1_matrix_mult.c -o p1
(base) gagan74@gaganlaptop:/mnt/d/Semester_06/MPC/LAB/07_lab_$ ./p1
Sequential Execution Time = 0.000001 seconds
Parallel Execution Time = 0.007765 seconds

Result Matrix:
0 14 20 42
0 20 40 60
0 26 52 78
0 32 64 96
(base) gagan74@gaganlaptop:/mnt/d/Semester_06/MPC/LAB/07_lab_$ gcc -fopenmp q2_matrix_collapse.c -o p2
(base) gagan74@gaganlaptop:/mnt/d/Semester_06/MPC/LAB/07_lab_$ ./p2
Execution Time with collapse(2) = 0.103014 seconds
Sample Output C[0][0] = 500
(base) gagan74@gaganlaptop:/mnt/d/Semester_06/MPC/LAB/07_lab_$ gcc -fopenmp q3_quicksort.c -o p3
(base) gagan74@gaganlaptop:/mnt/d/Semester_06/MPC/LAB/07_lab_$ ./p3
Original Array:
12 5 8 19 3 15 7 1 10 6 2 11

Sorted Array:
(base) gagan74@gaganlaptop:/mnt/d/Semester_06/MPC/LAB/07_lab_$
```

Program 03 :

```
#include <stdio.h>
#include <omp.h>

void swap(int *a, int *b)
{
    int temp;

    temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];

    int i = low - 1;

    int j;

    for(j = low; j < high; j++)
    {
        if(arr[j] < pivot)
        {
            i++;

            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);

    return i + 1;
}

void quicksort(int arr[], int low, int high)
{
    if(low < high)
    {
        int pi = partition(arr, low, high);

        #pragma omp task
        quicksort(arr, low, pi - 1);

        #pragma omp task
```

```

        quicksort(arr, pi + 1, high);

        #pragma omp taskwait
    }
}

int main()
{
    int arr[12] = {12, 5, 8, 19, 3, 15, 7, 1, 10, 6, 2, 11};

    int i;

    printf("Original Array:\n");

    for(i = 0; i < 12; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");

    #pragma omp parallel
    {
        #pragma omp single
        {
            quicksort(arr, 0, 11);
        }
    }

    printf("\nSorted Array:\n");

    for(i = 0; i < 12; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}

```

```

(base) gagan747@gaganLaptop:/mnt/d/Semester_06/HPC/LAB/07_lab_$ ./p3
Original Array:
12 5 8 19 3 15 7 1 10 6 2 11

```

```

Sorted Array:
(base) gagan747@gaganLaptop:/mnt/d/Semester_06/HPC/LAB/07_lab_$

```

Analysis of the code:

"In this program, QuickSort is implemented using OpenMP task-based parallelism. The array is divided into subarrays using the partition function, and each subarray is sorted recursively.

Using #pragma omp task, the left and right subarrays are processed in parallel. The taskwait directive ensures synchronization between recursive calls.

This approach improves performance by utilizing multiple threads to handle independent subproblems simultaneously."